Utica University | Department of Computer Science | Fall 2026
CSC 618: Information Retrieval | Week 10 Assignment

Submission 1: Evaluating Information Retrieval Systems (Part A)

Shaurav Parajuli| November 2026

1. Self-study topics

1. Why retrieval systems need evaluation. A ranking function has many choices behind it: stemming, weighting, length normalisation, and so on. A few good-looking searches cannot show whether a change helps overall, and it can improve some queries while harming others. Evaluation replaces impressions with repeatable measurements, so that two systems or two versions can be compared on the same task.

2. Relevance judgments and ground truth. A test collection has documents, queries and judgments (qrels) recording which documents meet each information need. Relevance is judged against the need, not the query's words, so a document can match every keyword and still be useless. Judgments reflect human opinion and assessors can disagree. Large collections cannot be fully judged, so TREC-style pooling judges only the top results of the participating systems, and unjudged documents are usually treated as non-relevant.

3. Precision, Recall and F1. These treat results as an unordered set. Precision asks how much of what was returned is useful; Recall asks how much of what is useful was returned. They pull against each other, since returning more documents tends to raise recall and lower precision. F1, their harmonic mean, stays low if either one is low, so it cannot be inflated by excelling at only one.

4. Precision@K and Reciprocal Rank. Users mostly read the top of the list. P@K counts relevant results in the top K, ignoring their order within K and anything below it. RR looks only at the first relevant result, which suits tasks where one correct answer is enough, such as finding a known page.

5. AP and MAP. AP takes precision at each rank where a relevant document appears and averages over all relevant documents, counting those never retrieved as 0. It therefore rewards both finding documents and placing them early. MAP averages AP across queries, giving each query equal weight.

6. Graded relevance and nDCG. Binary labels treat a perfect document and a marginal one alike. DCG adds up each document's grade after discounting it by rank. Dividing by the DCG of the ideal ordering gives nDCG in $[0,1]$, which makes queries comparable even when they have different numbers of relevant documents or grades.

7. Why metrics can disagree. Each metric represents a different user. RR cares about one answer, P@K about one screen of results, Recall about completeness, and nDCG about getting the best documents first. A system may win on RR by placing one good result first, then lose on MAP because the remaining relevant documents are buried. The right question is therefore "better for which task?"

2. Metric definitions

Rel is the relevant set, Ret the retrieved set, $\mathrm{rel}(k) \in \{0,1\}$ marks a relevant document at rank k, g_k is its grade, and Q the set of queries.

Table 1: Metric formulas.

Precision	$\frac{\lvert\mathrm{Rel}\cap\mathrm{Ret}\rvert}{\lvert\mathrm{Ret}\rvert}$	Recall	$\frac{\lvert\mathrm{Rel}\cap\mathrm{Ret}\rvert}{\lvert\mathrm{Rel}\rvert}$
F1P@Krel(k)	$\frac{1}{-\sum_{k=1}^{K}\frac{2PR}{P+R}}$		
RR	$\frac{1}{\text{rank of first relevant}}$ (0 if none)	MRR	$\frac{1}{\lvert Q\rvert}\sum_q \mathrm{RR}_q$
AP	$\frac{1}{\lvert\mathrm{Rel}\rvert}\sum_k P@k\cdot\mathrm{rel}(k)$	MAP	$\frac{1}{\lvert Q\rvert}\sum_q \mathrm{AP}_q$
DCG@K	$\sum_{k=1}^{K}\frac{g_k}{\log_2(k+1)}$	nDCG@K	$\frac{\mathrm{DCG@K}}{\mathrm{IDCG@K}}$

3. Worked example

The assignment does not supply a separate example, so I use query Q1 from Part B. Relevant set = {D1, D3, D5}; grades D1=3, D3=2, D5=1. The two ranked lists are:

Table 2: Q1 ranked lists (R = relevant, N = non-relevant).

Rank	1	2	3	4	5
System A	D1 (R, 3)	D2 (N, 0)	D3 (R, 2)	D5 (R, 1)	D4 (N, 0)
System B	D2 (N, 0)	D1 (R, 3)	D4 (N, 0)	D3 (R, 2)	D6 (N, 0)

Table 3: Calculations.

Metric	System A	System B
Precision	3/5 = 0.6000	2
Recall	3/3 = 1.0000	2
F1	2(0.6)(1)/(1.6)= 0.7500	2
P@3	2/3 = 0.6667	$/5 = 0.4000$ $/3 = 0.6667$ $(0.4)(0.6667)/(1.0667) = 0.5000$ $/3 = 0.3333$ $/2 = 0.5000$ $(1/2+2/4)/3 = 0.3333$ $\frac{3}{\log_2 3}+\frac{2}{\log_2 5} = 2.7541$
RR	1/1 = 1.0000	1

AP	(1/1+2/3+3/4)/3 = 0.8056	
DCG@5	$3+\frac{2}{\log_2 4}+\frac{1}{\log_2 5}=4.4307$	
IDCG@5	$3+\frac{2}{\log_2 3}+\frac{1}{\log_2 4}=4.7619$	4.7619 (same ideal list)
nDCG@5	4.4307/4.7619 = 0.9305	2.7541/4.7619 = 0.5784

Interpretation. System A beats B on every metric for Q1. It finds all three relevant documents, opens with the best one and loses little to the ideal order: only the non-relevant D2 at rank 2 keeps it from a perfect nDCG. B misses D5 and places a non-relevant document first, so a user sees a wrong result before anything useful. These values match the program output in Submission 2.

4. Summary of the study resources

Resource	Summary
Manning, Raghavan and Schütze, *IR*, Ch. 8 [1]	The textbook foundation. It sets out what a test collection needs, covers unranked measures and then ranked ones (MAP, P@K, NDCG), explains how reliable judgments are built through pooling and assessor-agreement checks, and reminds readers that relevance only approximates user happiness, which also depends on things like speed and interface.
Video: Evaluation Measures for Search and Recommender Systems [2]	A practical walkthrough of Recall@K, MRR, MAP@K and NDCG@K with Python on a tiny eight-image example. Main points: offline metrics come before deployment, and online tests such as A/B tests still need to back them up; Recall@K ignores order and can be inflated with a large *K*; DCG needs normalising because it has no fixed range; and graded labels are more costly to obtain.
NIST TREC data [3,4]	The benchmark source for the field. Each collection has documents, topics and relevance judgments. Topics and qrels can generally be downloaded directly, but most document sets are copyrighted and must be licensed. This is why I used the openly available Cranfield collection in Part C.

;

5. Conclusion

Systematic evaluation is necessary because search quality cannot be judged reliably from a few example queries or from the developer's own intuition. Fixed queries, documents and judgments isolate the effect of the system;

well-chosen metrics state which user need is being measured; and multiple queries with significance testing show whether a difference is likely to hold beyond chance. Part C of this assignment gave a concrete example: BM25 and TF-IDF looked different on individual queries, but the mean difference across 225 queries was not statistically significant. Offline results still have limits, so they should guide, not replace, testing with real users.

References

[1] C. D. Manning, P. Raghavan, H. Schütze, *Introduction to Information Retrieval*, Cambridge Univ. Press, 2008, Ch. 8. https://nlp.stanford.edu/IR-book/html/htmledition/evaluation-in-informationretrieval-1.html

[2] "Evaluation Measures for Search and Recommender Systems" (video). https://www.youtube.com/watch?v=BD9TkvEsKwM

[3] NIST TREC, Data. https://trec.nist.gov/data.html

[4] NIST TREC, Frequently Asked Questions. https://trec.nist.gov/faq.html

[5] K. Järvelin and J. Kekäläinen, "Cumulated gain-based evaluation of IR techniques," *ACM TOIS*, 20(4), 2002.